Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 6

Disciplina: Tehnici de programare

Tema: ” Informatii invisibile: forensincs si steganografie ”

A efectuat : st. gr.SI – 262 Bodaci Bogdan

A verificat : asistent universitar Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

Pregătire (comună celor două lucrări)

Intră în depozitul disciplinei, activează mediul virtual și instalează biblioteca de imagini. Apoi rulează scriptul de pregătire de mai jos: el îți creează, chiar la tine, fișierele pe care le vei analiza (un șir codat în straturi, un mesaj XOR, o poză cu EXIF și GPS, și o poză cu un fișier ascuns). Astfel toată lumea lucrează pe aceleași ținte, repetabil.

```
cd tehnici-prog-<nume>
# activeaza mediul (Windows PowerShell):
.\.venv\Scripts\Activate.ps1
# sau Linux:  source .venv/bin/activate

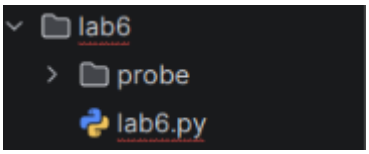
pip install pillow
mkdir -p arsenal/forensics teste probe

# creeaza fisierele de laborator:
python probe/pregateste.py      # scrie in probe/
```

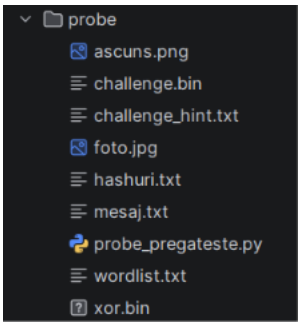
Scriptul probe/pregateste.py îți este dat (în depozitul de start al laboratorului). El produce: probe/mesaj.txt (șir în mai multe straturi de codare), probe/xor.bin (text criptat cu XOR pe un octet), probe/hashuri.txt (câteva hash-uri de spart), probe/foto.jpg (cu EXIF și GPS) și probe/ascuns.png (cu o arhivă lipită la coadă). Dacă lipsește, cere-l asistentului.

Ce poate crăpa. Dacă `import PIL` dă eroare, biblioteca se numește la instalare `pillow` (nu „PIL"). Dacă nu vezi `(.venv)` în prompt, mediul nu e activat.

Deci am create un directory nou(lab6) si am create un directory nou(probe) in el si l-am numit probe, la fel si are arsenal/forensics dupa care am dat (python probe/pregateste).




```
(.venv) PS C:\Users\Akicide\Desktop\Pycharm\lab6> python3 .\probe\probe_pregateste.py
  scris: mesaj.txt
  scris: xor.bin
  scris: hashuri.txt
  scris: wordlist.txt
  scris: foto.jpg
  adaugat ZIP la coada: ascuns.png
  scris: challenge.bin
  scris: challenge_hint.txt
Gata. Tintele de laborator sunt in: C:\Users\Akicide\Desktop\Pycharm\lab6\probe
```



Aici sunt toate filurile care mi-au aparut in urma rularii.

Mereu folosesc aceasta comanda de run in terminal, fara ea nu lucreaza:

```
python ./lab6/arsenal/forensics/lab6.py "NTM1OTU3NTk="
```

A1.Text si octeti:incalzirea

Aici este programul care merge:

```
s = "Salut"
b = s.encode()          # bytes
print(b, b[0])          # b"Salut" 83
# TODO: tipareste b.hex() si refa bytes din hex
#       cu bytes.fromhex(...)
h = b.hex()
print(h)
b2 = bytes.fromhex(h)
print(b2)
```

Aici este outputul:

```
C:\Users\Akici
b'Salut' 83
53616c7574
b'Salut'
```

A2.Recunosti stratul dupa forma

Aici e programul care functioneaza:

```
import re
def ghici_strat(s :{__contains__, __len__} ):  2 usages
        # TODO: daca are %xx -> "url"
    if "%" in s:
            return "url"
        # TODO: daca doar 0-9 a-f si lungime para -> "hex"
    if len(s) % 2 == 0 and all(c in
        "0123456789abcdefABCDEF" for c in s):
            return "hex"
        # TODO: daca doar A-Za-z0-9+/= -> "base64"
    if all(c in
        "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/=" for c in s):
            return "base64"
    return "necunoscut"
print(ghici_strat("%2Fetc"))   # url
print(ghici_strat("48656c6c6f"))  # hex
```

Aici e outputul:

```
C:\Users\Akicide\Desktop\
b'Salut' 83
53616c7574
b'Salut'
url
hex
```

A3.Desfaci stratul recunoscut

Aici e codul care merge:

```python
from urllib.parse import unquote
def desfa(s :{__contains__, __len__} ):  1 usage
    strat = ghici_strat(s)
    # TODO: pentru fiecare strat, decodeaza
    #   base64 -> base64.b64decode(s)
    #   hex    -> bytes.fromhex(s)
    #   url    -> unquote(s).encode()
    # intoarce (strat, rezultat_bytes)
    if strat == "base64":
        rezultat = base64.b64decode(s)
    elif strat == "hex":
        rezultat = bytes.fromhex(s)
    elif strat == "url":
        rezultat = unquote(s).encode()
    else:
        rezultat = s.encode()
    return strat, rezultat
```

Aici e outputul:

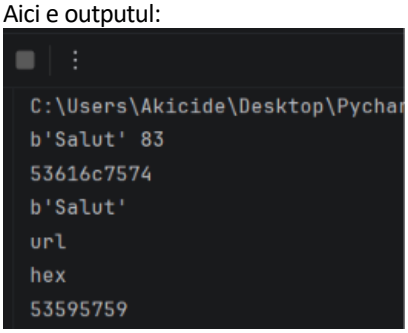
A4.Straturi multiple, intr-o bucla

Aici e codul care merge:

```python
def citibil(b):  1 usage
    # TODO: True daca toti octetii sunt 32..126
    return all(32 <= c < 127 for c in b)


val = "NTM1OTU3NTk="
for _ in range(8):
    strat, b = desfa(val)
    # TODO: daca citibil(b): opreste si tipareste
    #       altfel val = b.decode(); continua
    if citibil(b):
        print(b.decode())
        break
    val = b.decode()
```

Aici e outputul:



A5.Spargi un XOR cu o cheie de un octet

Aici e codul care merge:

```
date = open("probe/xor.bin", "rb").read()
for k in range(256):
    clar = bytes(b ^ k for b in date)
    # TODO: daca citibil(clar):
    #       print("cheie:", k, clar.decode())
    if citibil(clar):
        print("cheie:",k,clar.decode())
```

Aici e outputul:

```
cheie: 128 Un singur octet tine tot secretul... gaseste-l! FLAG{xor_un_octet}
cheie: 129 To!rhofts!nbudu!uhod!unu!rdbsdutm///!f`rdrud,m !GM@Fzyns^to^nbudu|
cheie: 131 Vm#pjmdvq#l`wfw#wjmf#wlw#pf`qfwvo---#dbpfpwf.o*#EOBDx{lq\vm\l`wfw~
cheie: 133 Pk%vlkbpw%jfq`q%qlk`%qjq%v`fw`qpi+++%bdv`vq`(i$%CIDB~}jwZpkZjfq`qx
cheie: 134 Sh&uohast&iercr&rohc&rir&ucetcrsj(((&agucurc+j'&@JGA}~itYshYiercr{
cheie: 136 ]f({afo}z(gk|m|(|afm(|g|({mkzm|}d&&&(oi{m{|m%d)(NDIOspgzW}fWgK|m|u
cheie: 137 \g)z`gn|{)fj}l})}`gl)}f})zlj{l}|e''')nhzlz}l$e()OEHNrqf{V|gVfj}l}t
cheie: 142 [`.}g`i{|.amzkz.zg`k.zaz.}km|kz{b   .io}k}zk#b/.HBOIuva|Q{`Qamzkzs
cheie: 143 Za/|fahz}/`l{j{/{faj/{`{/|jl}j{zc!!!/hn|j|{j"c./ICNHtw`}PzaP`l{j{r
cheie: 146 G|2a{|ug`2}qfwf2f{|w2f}f2awq`wfg~<<<2usawafw?~32T^SUij}`Mg|M}qfwfo
cheie: 148 Az4g}zsaf4{w`q`4`}zq4`{`4gqwfq`ax:::4sugqg`q9x54RXUSol{fKazK{w`q`i
cheie: 149 @{5f|{r`g5zvapa5a|{p5aza5fpvgpa`y;;;5rtfpfap8y45SYTRnmzgJ`{Jzvapah
cheie: 151 By7d~ypbe7xtcrc7c~yr7cxc7drtercb{9997pvdrdcr:{67Q[VPloxeHbyHxtcrcj
cheie: 153 Lw9jpw~lk9vzm|m9mpw|9mvm9j|zk|mlu7779~xj|jm|4u89_UX^bavkFlwFvzm|md
cheie: 155 Nu;hru|ni;txo~o;oru~;oto;h~xi~onw555;|zh~ho~ow:;]WZ\`ctiDnuDtxo~of
cheie: 157 Hs=ntszho=r~ixi=itsx=iri=nx~oxihq333=z|nxnix0q<=[Q\ZferoBhsBr~ixi`
cheie: 159 Jq?lvqxjm?p|kzk?kvqz?kpk?lz|mzkjs111?x~lzlkz2s>?YS^Xdgpm@jq@p|kzkb
```

A6.Spargi un hash prin dictionar

Aici e codul care merge:

```python
import hashlib
tinta = open("probe/hashuri.txt").readline().strip()
print(len(tinta))   # 32 md5 / 40 sha1 / 64 sha256
for cuv in open("probe/wordlist.txt", encoding="latin1"):
    cuv = cuv.strip()
    # TODO: daca hashlib.md5(cuv.encode()).hexdigest()
    #       == tinta: print("gasit:", cuv); break
    if hashlib.md5(cuv.encode()).hexdigest() == tinta:
        print("gasit:", cuv)
        break
```

Aici e outputul:

```
32
gasit: password
```

A7.Argparse, fisa JSON si un test

Aici e codul care merge:

```python
import argparse, json, os
p = argparse.ArgumentParser()
p.add_argument("intrare")          # sir sau cale
p.add_argument(*name_or_flags: "--fisier", action="store_true")
a = p.parse_args()
# TODO: daca a.fisier, citeste octetii din cale;
#       altfel trateaza intrarea ca sir
# TODO: salveaza {strat, rezultat} in
#       arsenal/forensics/fisa.json
if a.fisier:
    with open(a.intrare, "rb") as f:
        val = f.read().decode().strip()
else:
    val = a.intrare


strat, b = desfa(val)
os.makedirs(name: "../arsenal/forensics", exist_ok=True)
with open("../arsenal/forensics/fisa.json", "w") as f:
    json.dump(obj: {"strat": strat, "rezultat": b.decode(errors="replace")}, f)
```

Aici e outputul:

6.py {} fisa.json ×

```
{"strat": "base64", "rezultat": "53595759"}
```

Lucrarea B

B1.Magic bytes: tipul real

Aici e codul care lucreaza:

```
def tip_real(cale):  1 usage
    cap = open(cale, "rb").read(8)
    # TODO: daca cap incepe cu b"%PDF" -> "pdf"
    #       b"\x89PNG" -> "png" ; b"\xff\xd8" -> "jpeg"
    #       b"PK\x03\x04" -> "zip/docx"
    if cap.startswith(b"%PDF"):
        return"pdf"
    if cap.startswith(b"x89PNG"):
        return"png"
    if cap.startswith(b"\xff\xd8"):
        return"jpeg"
    if cap.startswith(b"PK\x03\x04"):
        return"zip/docx"
    return cap.hex()
print(tip_real("lab6/probe/foto.jpg"))
```

Aici e outputul:

```
jpeg
```

B2.strings: textul citibil dintr-un binar

Aici e codul care merge:

```
import re
date = open("lab6/probe/ascuns.png", "rb").read()
# TODO: for m in re.findall(rb"[ -~]{4,}", date):
#           print(m.decode())
# cauta un indiciu: o cale, un mesaj, un nume de fisier

for m in re.findall( pattern: rb"[ -~]{4,}", date):
    print(m.decode())
```

Aici e outputul:

```
IHDR
IDATx
IEND
KTf]
secret.txtFLAG{ascuns_prin_carving}
KTf]
secret.txtPK
```

B3.Metadate EXIF si GPS din date

Aici e codul care lucreaza:

```
from PIL import Image
from PIL.ExifTags import TAGS

exif = Image.open("lab6/probe/foto.jpg")._getexif() or {}
gps = exif.get(34853, {})
for id, val in exif.items():
    print(TAGS.get(id, id), "=", val)
# TODO: ia blocul GPS (tag 34853) si tipareste
#       latitudinea si longitudinea

    print("Latitudinea:", gps.get(2))
    print("Longitudinea:", gps.get(4))
```

Aici e outputul:

```
Model = iPhone 13
Latitudinea: (47.0, 1.0, 0.0)
Longitudinea: (28.0, 51.0, 27.0)
DateTime = 2026:09:14 10:30:00
Latitudinea: (47.0, 1.0, 0.0)
Longitudinea: (28.0, 51.0, 27.0)
GPSInfo = {1: 'N', 2: (47.0, 1.0, 0.0), 3: 'E', 4: (28.0, 51.0, 27.0)}
Latitudinea: (47.0, 1.0, 0.0)
Longitudinea: (28.0, 51.0, 27.0)
Make = Apple
Latitudinea: (47.0, 1.0, 0.0)
Longitudinea: (28.0, 51.0, 27.0)
```

B4.File carving: scoti fisierul ascuns

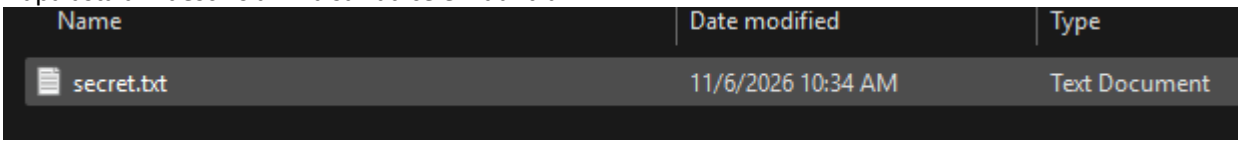
Aici e codul care lucreaza:

```
date = open("labó/probe/ascuns.png", "rb").read()
i = date.find(b"PK\x03\x04")
# TODO: daca i != -1:
#   open("gasit.zip","wb").write(date[i:])
#   print("arhiva extrasa de la octetul", i)
#   apoi dezarhiveaza si vezi ce era inauntru
if i != 1:
    open("gasit.zip", "wb").write(date[i:])
    print("arhiva extrasa de la octetul", i)
```

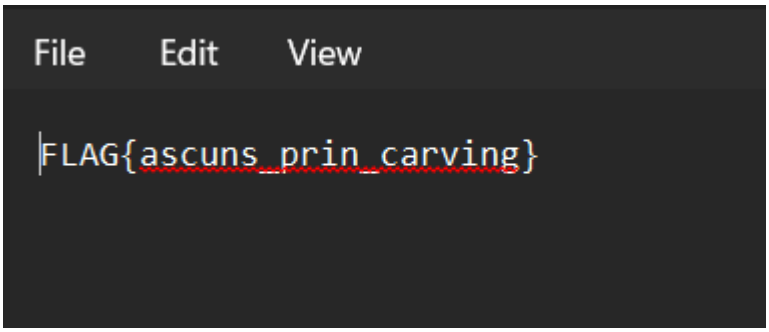
Aici e outputul:

```
arhiva extrasa de la octetul 782
```

Dupa asta am deschis arhiva sa vad ce e inauntru:



Inauntrul lui era:



B5.Asunzi un mesaj prin LSB

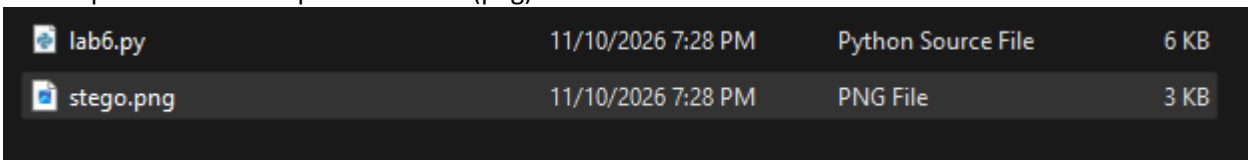
Aici e codul care lucreaza:

```python
from PIL import Image
img = Image.open("lab6/probe/foto.jpg").convert("RGB")
px = list(img.getdata())
mesaj = b"FLAG{ascuns_in_pixeli}"
biti = "".join(f"{x:08b}" for x in mesaj)
# TODO: pentru fiecare bit, ia un canal si pune
#   canal = (canal & ~1) | int(bit)
px_nou = px.copy()
for i,bit in enumerate [str] (biti):
    r, g, b = px_nou[i]
    r = (r & ~1) | int(bit)
    px_nou[i] = (r, g, b)
# TODO: img.putdata(px_nou); img.save("stego.png")
    img.putdata(px_nou)
    img.save("stego.png")
```

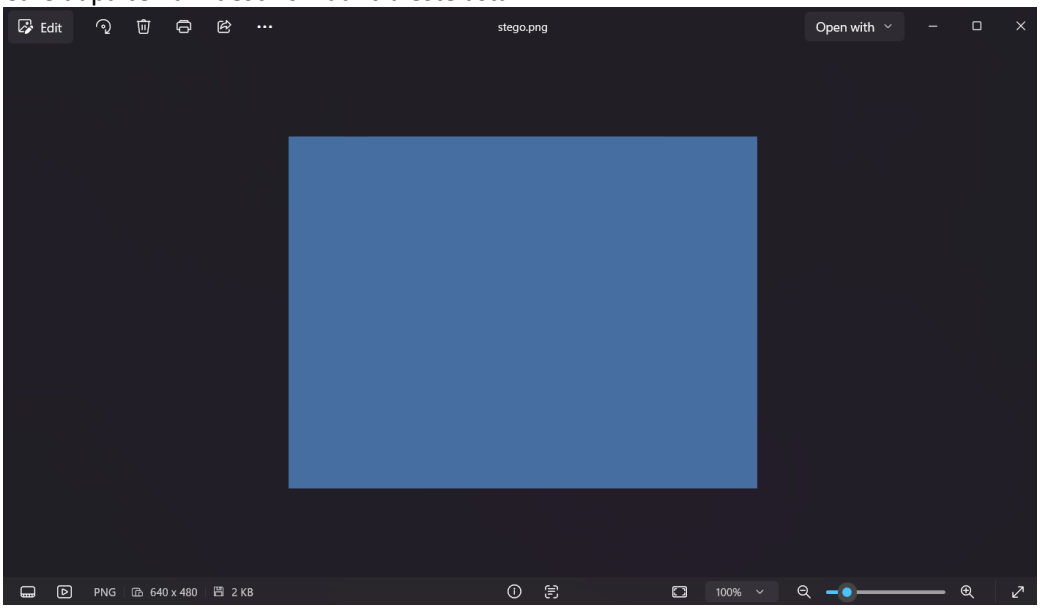
Aici e outputul:

```
C:\Users\Akicide\Desktop\Pycharm\lab6\arsenal\forensics\lab6.py:172: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use get_flattened_data instead.
  px = list(img.getdata())
```

Care dupa in folder mi-a aparut acest file (png):



Care dupa ce l-am deschis inauntru este asta:

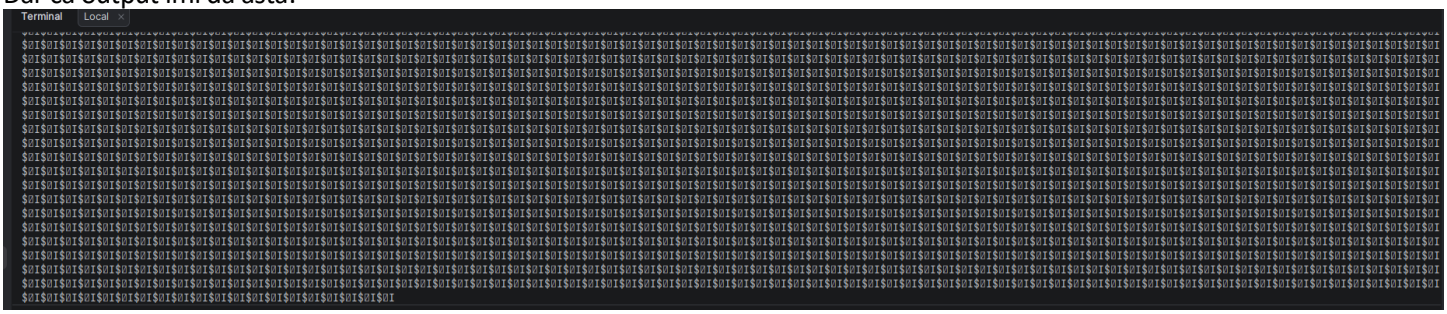


B6.Dezvalui mesajul inapoi

Am facut acest cod:

```
px = list(Image.open("stego.png").convert("RGB").getdata())
biti = ""
for (r, g, b) in px:
    # TODO: aduna ultimii biti (r&1, g&1, b&1)
    pass
    biti += str(r&1) + str(g&1) + str(b&1)
# TODO: grupeaza cate 8 biti -> octeti -> text
#       opreste-te la primul octet 0 (final de mesaj)
mesaj=""
for i in range(0, len(biti), 8):
    octet = int(biti[i:i+8], 2)
    if octet == 0:
        break
    mesaj += chr(octet)
print(mesaj)
```

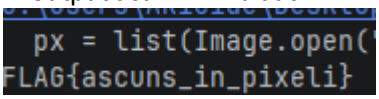
Dar ca output imi da asta:



Deci am schimbat putin:

```
px = list(Image.open("stego.png").convert("RGB").getdata())
biti = ""
for (r, g, b) in px:
    # TODO: aduna ultimii biti (r&1, g&1, b&1)
    pass
    biti += str(r&1)
# TODO: grupeaza cate 8 biti -> octeti -> text
#       opreste-te la primul octet 0 (final de mesaj)
mesaj=""
for i in range(0, len(biti), 8):
    mesaj += chr(int(biti[i:i+8], 2))
print(mesaj)
```

In output acum mi—a dat:



Concluzie

În urma finalizării acestui laborator de forensics și steganografie, am reușit să construiesc și să testez un arsenal complet de unelte în Python pentru analiza și decodarea datelor.

Principalele obiective atinse în cadrul lucrării sunt:

Dezvoltarea decoderului universal (Lucrarea A): Am implementat funcționalități pentru detectarea automată și decodarea straturilor multiple (Base64, Hex, URL), spargerea prin forță brută a unui cifru XOR pe un octet și recuperarea parolelor din hash-uri (MD5/SHA) folosind un dicționar, completate cu argumente CLI, salvare în format JSON și teste unitare.

Analiza forensică și steganografia (Lucrarea B): Am analizat semnăturile binare reale (magic bytes) ale fișierelor, am extras șirurile de text imprimabile și am extras metadatele EXIF, inclusiv coordonatele GPS, dintr-o imagine. De asemenea, am realizat file carving pentru a recupera o arhivă ascunsă și am aplicat tehnici de steganografie LSB (Least Significant Bit) pentru a ascunde și extrage un mesaj direct din pixelii unei imagini fără a-i altera aspectul vizual.